GlyphJS — Complete Feature Showcase

GlyphJS is a Markdown-to-interactive-UI rendering engine. You write ordinary Markdown enriched with `ui:` fenced code blocks, and the pipeline compiles everything into a polished, interactive document — locally, with no server, no build step, and no cloud dependency.

This document demonstrates two things in sequence: first, that GlyphJS renders all standard Markdown faithfully; second, that it extends Markdown with 28 interactive components covering data visualization, diagrams, content layout, and interactive widgets.

Markdown Reference

A systematic index of every CommonMark element. Use this section to see exactly how the active theme colors, spaces, and contrasts each element.

All heading levels

H1 — Primary accent color

H2 — Secondary palette color

H3 — Heading with accent left bar

H4 — Heading text color, no decoration

H5 — MUTED · UPPERCASE · TRACKED

H6 — SMALLEST · MUTED · UPPERCASE

Inline typography

Plain body text flows at normal weight. Within a sentence: **bold text** picks up the heading color, *italic text* stays in body color but slants, ***bold italic*** combines both, ~~strikethrough~~ draws a line through, and `inline code` switches to the monospace code palette with its own background tint.

You can nest freely: **bold containing** `monospace code`, *italic containing* ***nested bold***. Long italic runs look like this: *this entire sentence is set in italics to show the italic rendering across a fuller run of theme-colored body text — note the weight, slant angle, and color.*

Hard line breaks are two trailing spaces:
This line starts on a new line but belongs to the same paragraph block.

Escaped characters render literally: *not italic* `not code` [not a link] # not a heading.

Special HTML entities: & < > © — …

Unordered lists

- First top-level item
- Second item with **bold**, *italic*, and `code` inline
  - Nested item at depth 2 — note the accent-colored marker
  - Another nested item
    - Depth 3 — deeply nested, still themed
    - A second item at depth 3
  - Back to depth 2
- Third top-level item

Ordered lists

1. First step
2. Second step with a longer description that intentionally wraps to demonstrate how multi-line list items look in the current theme — does the leading align cleanly with the text?
3. Third step
   1. Sub-step one
   2. Sub-step two
      1. Sub-sub-step — three levels deep
4. Fourth step, back at the top level

Mixed nesting

- Category A
  1. First ordered item inside an unordered list
  2. Second ordered item
- Category B
  - Unordered sub-item
    1. Ordered inside unordered inside unordered
- Category C, no nesting

Blockquotes

> A simple single-paragraph blockquote. The background is `--glyph-surface`, the left border is `--glyph-accent`, and the text uses `--glyph-text-muted`.

> A multi-paragraph blockquote.The second paragraph continues here. **Bold** and *italic* and `code` work normally within blockquotes.— An attribution line in the same blockquote.

Nested blockquotes go one level deeper:

> Outer level of a nested quote. This sets up the context.

Code blocks

An inline span: `const ir = await compile(markdown)` — monospace, code background, code text color.

A TypeScript fenced block:

```
interface ThemeFileData {
  base: 'light' | 'dark';
  overrides: Record<string, string>;
}

function resolveThemeVars(
  base: ThemeFileData['base'],
  overrides: ThemeFileData['overrides'],
): Record<string, string> {
  const baseVars = base === 'dark' ? DARK_THEME_VARS : LIGHT_THEME_VARS;
  return { ...baseVars, ...overrides };
}
```

A shell script:

```
# Export to every theme in one pass
for theme in themes/*.yml; do
  name=$(basename "$theme" .yml)
  glyphjs export doc.md \
    --format pdf \
    --theme-file "$theme" \
    --continuous \
    --padding "2rem 2.5rem" \
    -o "demo/demo-${name}.pdf"
done
```

A plain fenced block with no language tag:

```
This is unformatted preformatted text.
Monospace font, code background, no syntax highlighting.
Useful for output samples, ASCII diagrams, or terminal dumps.
```

Tables

A standard table with a header row, body rows, and alternating stripe:

Variable	Role	Dark default	Light default
`--glyph-bg`	Page background	`#1a1a1a`	`#ffffff`
`--glyph-accent`	Primary accent	theme-defined	theme-defined
`--glyph-surface`	Raised surface	`#242424`	`#f5f5f5`
`--glyph-heading`	Heading text	`#f0f0f0`	`#1a1a1a`
`--glyph-text-muted`	Secondary text	`#888`	`#666`
`--glyph-border`	Subtle dividers	`#333`	`#e0e0e0`
`--glyph-code-bg`	Code background	`#111`	`#f0f0f0`
`--glyph-code-text`	Code text	`#e0e0e0`	`#333`

Horizontal rules

Thematic breaks created with `---`:

And with `***`:

Links

A regular link to the GlyphJS docs uses `--glyph-link` color with a soft underline tinted from `--glyph-accent-muted`. On hover it shifts to `--glyph-link-hover`.

An inline link with a title carries a tooltip title attribute.

Autolinks: https://glyphjs.dev and bare reference links are rendered the same way.

Images

Images are max-width constrained and corner-rounded. A missing image shows the alt text:

The GlyphJS pipeline: Markdown → Parser → Compiler → IR → Runtime → DOM

Standard Markdown

Everything in this section is plain CommonMark. GlyphJS passes it through unchanged, rendered by whatever host environment you're using — a browser, Obsidian, a PDF export, or a static site.

Text formatting

Paragraphs are separated by blank lines. Within a paragraph you can use **bold text** for emphasis, *italic text* for softer stress, `inline code` for identifiers and values, and ~~strikethrough~~ when you need to show something has been removed.

You can combine these freely: a `typed constant` or an *italicized phrase with a **nested bold** inside it*. GlyphJS does not restrict how Markdown inline syntax nests.

Lists

Unordered lists work with `-`, `*`, or `+` markers:

- Parser — transforms `ui:*` fenced blocks into AST nodes via a remark plugin
- Compiler — walks the AST and produces a typed JSON Intermediate Representation (IR)
- Runtime — a React rendering engine that takes the IR and produces interactive DOM
- Components — 28 built-in components, each with a Zod schema and a React renderer

Ordered lists use numeric prefixes:

1. Write a Markdown file with embedded `ui:` code blocks
2. Run `glyphjs export document.md --format pdf -o output.pdf`
3. Open the PDF — components are fully rendered, no JavaScript required

Lists can nest up to any depth:

- `@glyphjs/types` — shared TypeScript interfaces
  - `Block` — the fundamental unit of the IR
  - `GlyphIR` — the complete document representation
- `@glyphjs/schemas` — Zod schemas for all 28 components
  - Used for validation at parse time
  - Used for JSON Schema generation via `glyphjs schemas`

Blockquotes

> **Design principle:** A document format should compose with its host environment, not fight it. GlyphJS components are defined in plain YAML inside standard fenced code blocks — any Markdown editor that doesn't know GlyphJS shows the YAML source; any environment that does renders the component.

Blockquotes can hold any block-level content, including nested quotes:

> The pipeline is stateless and deterministic.

Code blocks

Fenced code blocks with language identifiers get syntax highlighting in most renderers:

```
import { compile } from '@glyphjs/compiler';

const ir = await compile(`
# My Document

\`\`\`ui:callout
type: tip
title: Hello world
content: GlyphJS compiled this.
\`\`\`
`);

console.log(ir.blocks[0].type); // "callout"
```

```
# A GlyphJS theme file — only override what you need.
# The base system fills in the rest.
base: dark
variables:
  --glyph-bg: '#1e1e2e'
  --glyph-accent: '#cba6f7'
  --glyph-heading: '#cdd6f4'
```

Tables

Markdown tables are rendered as styled HTML tables:

Package	Role	Key export
`@glyphjs/types`	Type definitions	`Block`, `GlyphIR`
`@glyphjs/schemas`	Validation	`componentSchemas`, `getJsonSchema`
`@glyphjs/compiler`	Compilation	`compile()`
`@glyphjs/runtime`	React rendering	`createGlyphRuntime()`
`@glyphjs/cli`	CLI tooling	`glyphjs` binary

Links and images

The GlyphJS documentation covers the full component reference, theming guide, and CLI usage. The source is at github.com/VledicFranco/glyphjs.

Horizontal rules

A thematic break separates sections with visual weight:

Everything above is standard Markdown. Everything below is rendered by GlyphJS.

GlyphJS Components

The following components are rendered by the GlyphJS runtime. Each `ui:` block is a YAML object validated against its Zod schema, compiled to IR, and mounted as a React component.

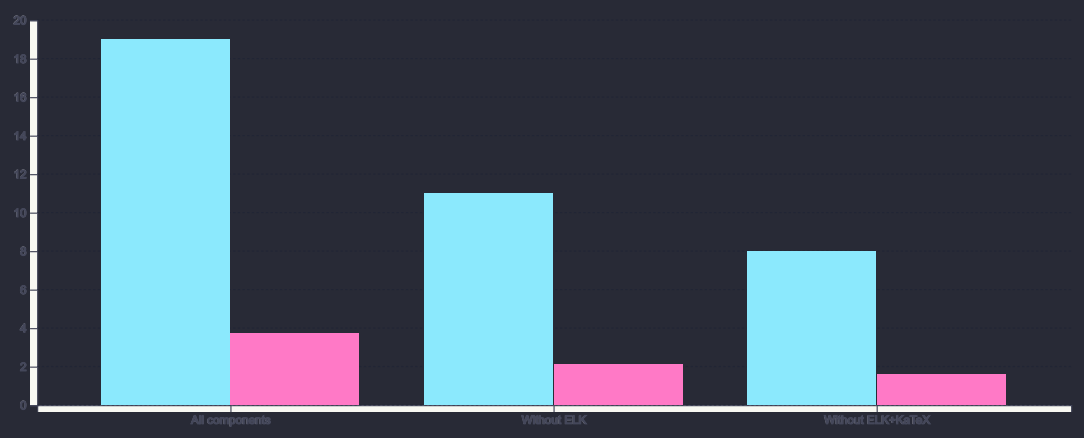
Callout

> 💡 **Plugin is working!**
> GlyphJS renders natively in Obsidian with full theme sync.

Table

Component	Status	Phase
Callout	✓ Rendering	1
Table	✓ Rendering	1
Chart	✓ Rendering	1
Live Preview	Pending	2

Chart



KPI

Components	Bundle (gzip)	Phase
28 total	**3.7** MB	**1** of 3

Steps

Phase 1 — Reading view

Bundle all packages. Register MarkdownPostProcessor. Mount React roots via MarkdownRenderChild. Ship with theme sync.

Phase 2 — Live preview

CM6 StateField with widget decorations. Cursor-aware block hiding. Debounced compile().

Phase 3 — Deep integration

Full theme mapping. Settings panel. Export-to-PDF command. Mobile support.

Timeline

TBD

Phase 2 — Live preview

CM6 StateField for in-editor component preview

Mar 1, 2026

Reading view working

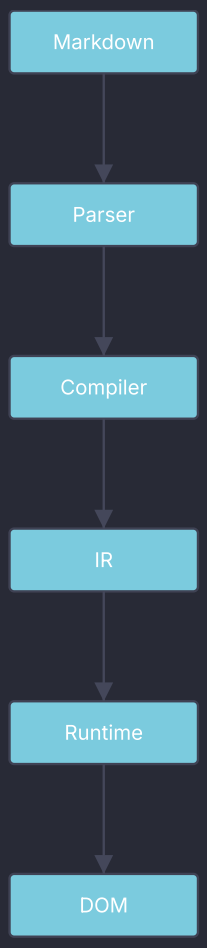
All 28 components rendering with Obsidian theme sync confirmed

Feb 28, 2026

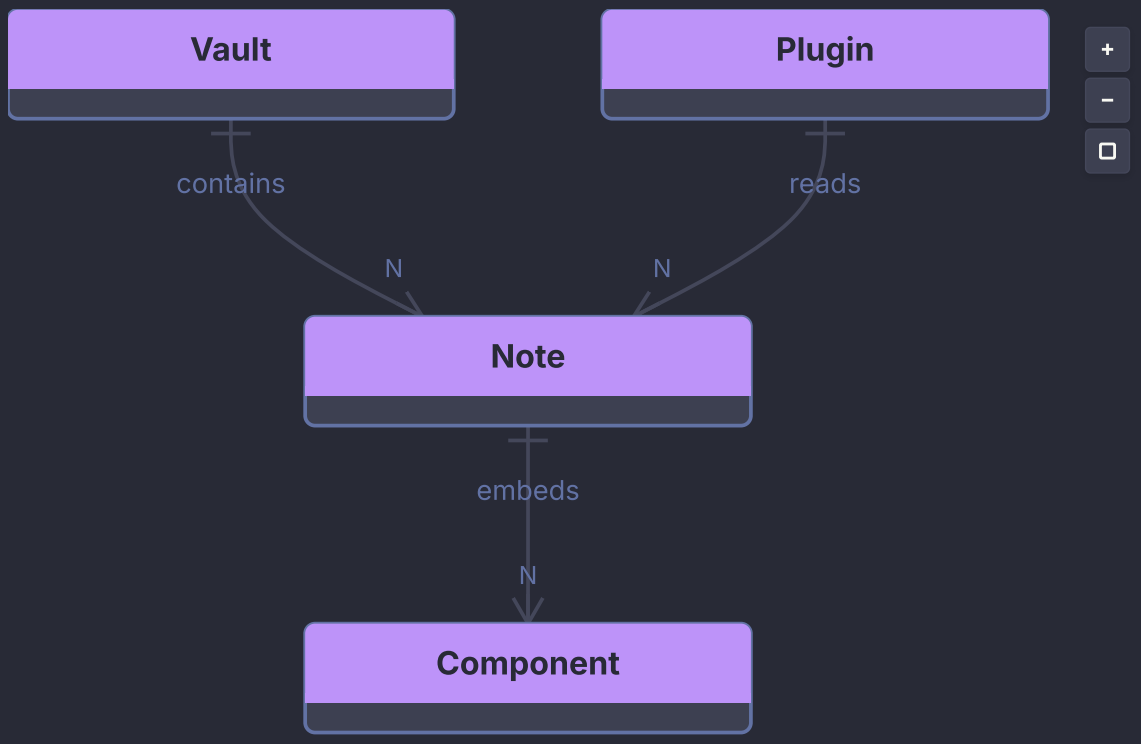
POC started

Created packages/obsidian-plugin in the glyphjs monorepo

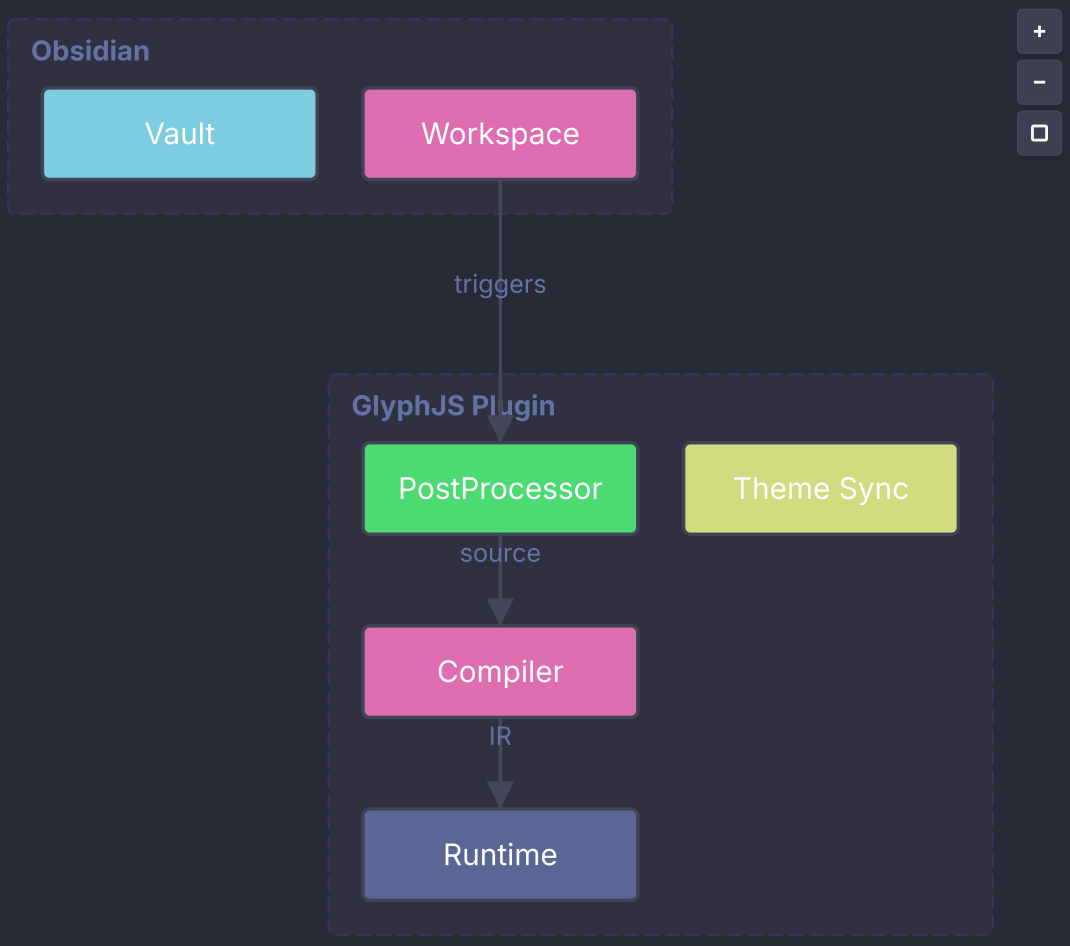
Graph



Relation



Architecture



Accordion

- Why not registerMarkdownCodeBlockProcessor?
- How does theme sync work?
- What about mobile?

Comparison

Integration approach

Feature	Bundled packages	CLI bridge
Works offline	✓	✗
Full interactivity	✓	✗
Community plugin eligible	✓	✗
Always latest version	✗	✓

Tabs

Overview | Pipeline | Theme

The GlyphJS Obsidian plugin renders interactive components natively in reading view. Uses `registerMarkdownPostProcessor` to scan rendered HTML for `ui:*` code blocks. Each block is compiled independently and mounted with React via `MarkdownRenderChild`.

Card

@glyphjs/compiler

@glyphjs/runtime

@glyphjs/components

Infographic

28	19 MB	3.7 MB	1.5.0
COMPONENTS	BUNDLE RAW	BUNDLE GZIP	MIN OBSIDIAN

CodeDiff

```diff
-  1  // Old approach - broken with colons
-  2  this.registerMarkdownCodeBlockProcessor(
-  3    "ui:callout",
-  4    (source, el, ctx) => {
-  5      ctx.addChild(new GlyphBlockChild(el, source));
+     1  // New approach - works reliably
+     2  this.registerMarkdownPostProcessor((el, ctx) => {
+     3    const codeBlocks = el.querySelectorAll('pre > code');
+     4    for (const code of codeBlocks) {
+     5      if (!code.className.startsWith('language-ui')) continue;
+     6      // extract type + source, mount React root
   6 7    }
-  7  );
+     8  });
   8 9
```

FileTree

```
packages
  obsidian-plugin
    src
      main.ts
      theme-sync.ts
    manifest.json
    esbuild.config.mjs
    styles.css
    package.json
```

Quiz

1. Which Obsidian API correctly handles ui:* language identifiers with colons?

- registerMarkdownCodeBlockProcessor
- registerMarkdownPostProcessor
- registerEditorExtension

2. What CM6 primitive is required for line-spanning widget decorations?

- ViewPlugin
- StateField
- EditorView.decorations

Kanban

TO DO 3	IN PROGRESS 1	DONE 2
Live preview (Phase 2)	Theme sync	Reading view
Mobile support		All 28 components
Settings panel		

Poll

Which GlyphJS component would you use most in Obsidian?

- Table
- Chart
- Callout
- Graph
- Kanban
- Architecture

Vote

Rating

Ease of setup

★ ★ ★ ★ ★

Documentation quality

★ ★ ★ ★ ★

Rendering performance

★ ★ ★ ★ ★

Theme integration

★ ★ ★ ★ ★

Ranker

1. Live preview
2. Obsidian theme sync
3. Mobile support
4. PDF export
5. Dataview interop

Slider

Max bundle size (MB) 0.5

0.5 10

Min Obsidian version (×10) 10

10 20

Matrix

	Reading View	Live Preview	Mobile	Total
Phase 1	0	0	0	0
Phase 2	0	0	0	0
Phase 3	0	0	0	0

Form

Plugin ID

glyphjs

Description

Render interactive GlyphJS components in Obsidian

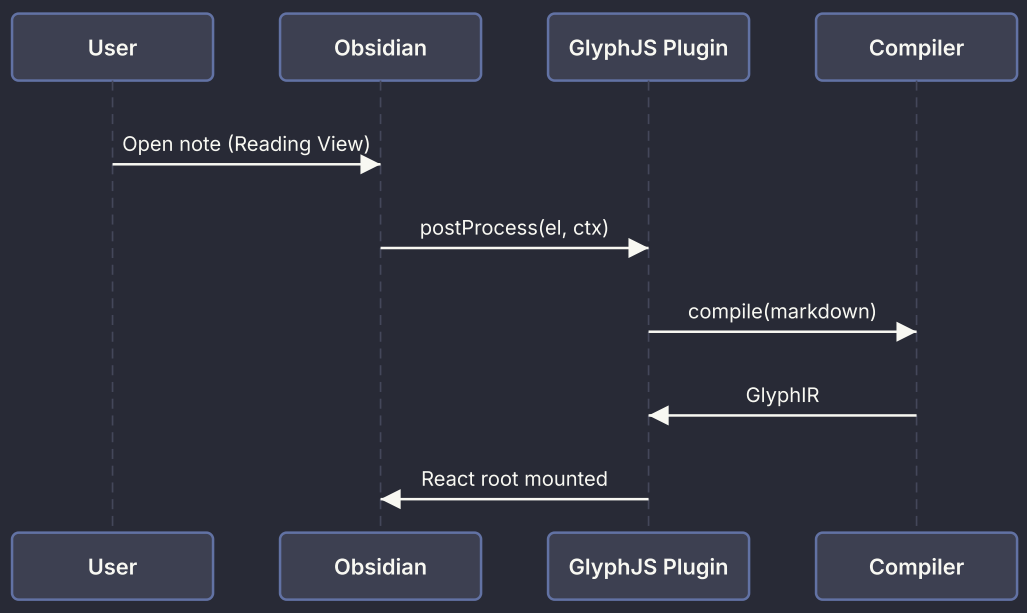
Current phase

Phase 1 — Reading view

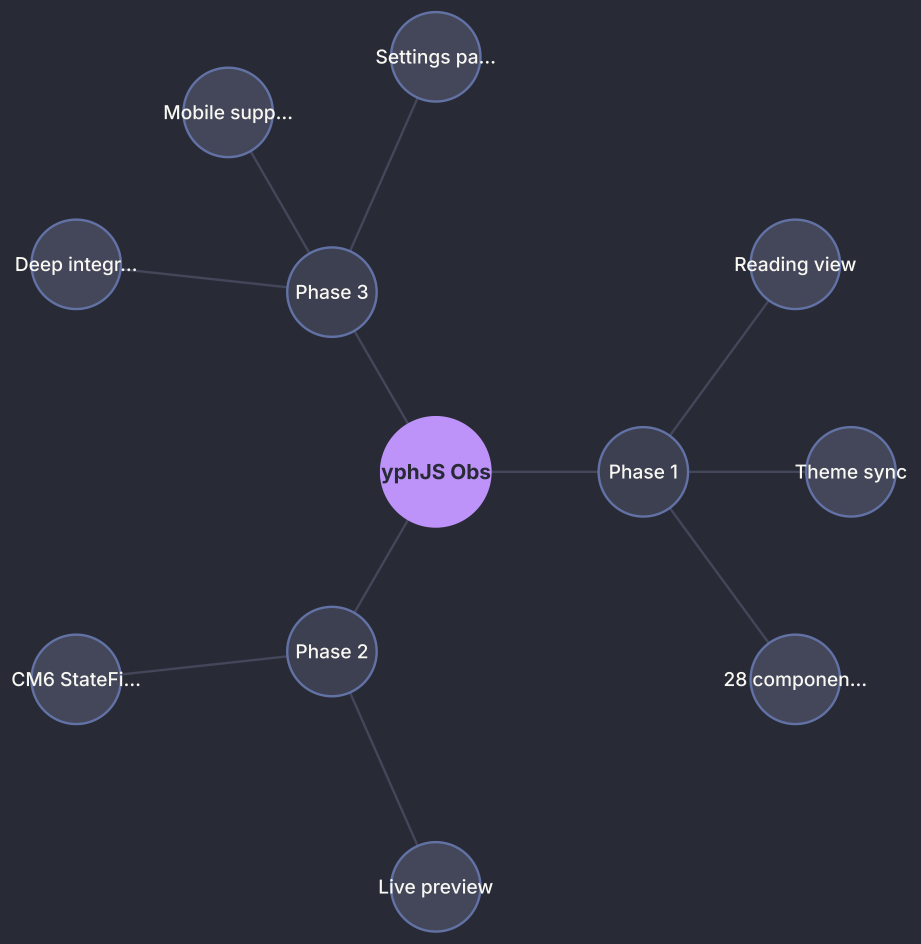
☐ Mobile support

Submit

Sequence



MindMap



Equation

$$E = mc^2$$

E = mc2

Mass-energy equivalence

Annotate

```
The plugin uses registerMarkdownPostProcessor to scan rendered HTML for
ui:* code blocks, then mounts React roots via MarkdownRenderChild for
proper lifecycle management.
```

ANNOTATIONS (0)

Select text to add annotations.